Introduction

When banks and lenders give out loans, they look at a lot of information to decide if they should give someone a loan and how much the loan should be. We're going to take a closer look at this process by using a dataset of people who have applied for loans. This dataset includes a variety of information such as whether the applicant is married, how many people depend on them financially, their level of education, whether they're self-employed, their income, and their credit history, among others. By analysing this data, we hope to predict how much money each person might receive as a loan.

To do this, we'll use a method called linear regression. Linear regression helps us understand the relationship between all the information we have (like income and education) and the loan amount. It's like drawing a line that best fits all the data points on a graph.

Here's how we'll do it:

1. Looking at the Data:

We'll start by looking at our data, seeing what it looks like, and checking for any patterns.

2. Cleaning the Data:

Sometimes, data can be messy — there might be missing information or mistakes. We need to clean it up so that our predictions are accurate. This means filling in missing values and making sure everything is in the right format. Even we can do cleaning and per- processing the data in the excel.

3. Splitting the Data:

We divide our data into two parts. One part is for teaching our computer model what to look for (this is called the training set), and the other part is for testing how well our model works (this is the test set).

4. Scaling the Data:

We need to make sure that no single piece of information outweighs the others. For example, someone's income might be in thousands, while the number of dependents might be just 1 or 2. We scale the data to prevent income from dominating our analysis just because its numbers are bigger.

5. Training the Model:

With our clean and scaled data, we teach our model to make predictions. We use the training set for this.

6. Making Predictions:

After our model has learned from the training data, we test it using the test set. We're trying to predict how much money the bank would loan to these applicants.

7. Visualizing:

It's easier to understand complex things when we can see them, so we'll create some graphs. These will show us the relationship between what we know about the applicants and the loan amounts our model predicts they should get.

8. Understanding Our Model:

After we make predictions, we'll talk about what they mean. We'll see which pieces of information (like income or education) are the best at predicting loan amounts, and we might find some surprises along the way!

9. Thinking About the Future:

Finally, we'll think about how we can use what we've learned. Maybe banks can use this model to help decide on loans, or perhaps it can help make sure everyone gets a fair offer.

By following these steps, we can build a model that helps us understand the loan process better and make predictions about loan amounts. This can help banks give out loans more fairly and help customers understand what affects their loan offers.

Data Description

This dataset contains loan application data. Each record provides multiple attributes associated with the loan application. Here are the descriptions of the columns present in the dataset:

Loan_ID: A unique identifier for the loan application.

Gender: The gender of the applicant (Male/Female).

Married: Marital status of the applicant (Yes/No).

Dependents: The number of individuals dependent on the applicant (0/1/2/3+).

Education: The educational level of the applicant (Graduate/Not Graduate).

Self_Employed: Indicates whether the applicant is self-employed (Yes/No).

ApplicantIncome: The income of the applicant.

CoapplicantIncome: The income of the co-applicant.

LoanAmount: The loan amount in thousands that the applicant has requested.

Loan_Amount_Term: The term (in days) over which the loan will be repaid.

Credit_History: A record of past credit use, represented as a binary number where 1 stands for a good credit history and 0 for a bad one.

Property_Area: The type of location where the property is situated (Urban/Semiurban/Rural).

Loan_Status: The status of the loan (Y/N), where 'Y' indicates that the loan was approved and 'N' indicates it was not.

The data seems to be comprehensive, covering various aspects that banks typically consider when assessing loan applications. It includes both categorical and numerical data types, offering a rich dataset for analysis and prediction of loan approval outcomes.

Slp:

The slope of the peak exercise ST segment

(0: upsloping,

1: flat,

2: downsloping)

We obtained a "Downsloping" trend from the linear regression model, this indicates a negative relationship between the 'ApplicantIncome' and the 'LoanAmount'. In simple terms, as the 'ApplicantIncome' increases, the 'LoanAmount' tends to decrease, according to the model's predictions. Let's delve into what this might imply and how to approach this outcome.

Explanation of the Output

Negative Slope (Downsloping Trend):

The negative slope suggests that higher-income applicants are predicted to receive lower loan amounts, which might seem counterintuitive at first glance. However, this can happen due to various reasons in real-world scenarios, which might be reflected in the data.

Possible Reasons for the Trend:

Debt-to-Income Ratio: Higher-income applicants might already have significant existing debts, leading to lower new loan amounts.

Loan Purpose and Risk Assessment: The loan amount could be influenced by the purpose of the loan and the associated risk. Higher-income individuals might apply for loans with lower amounts.

Data Distribution and Outliers: If high-income applicants are few or if their loan amounts are unusually low, this could skew the trend.

Approach

1. Data Preparation

Data Loading: We started by loading two datasets: one for training and another for testing. These datasets contain information related to loan applicants, including various features such as applicant income, coapplicant income, loan amount term, credit history, and categorical variables like gender, marital status, education, self-employment, and property area.

Data Cleaning: Rows with missing values were removed from both the training and test datasets to ensure data quality and consistency.

2. Feature Engineering

Feature Selection: We identified numerical and categorical features in the dataset to guide the preprocessing steps.

3. Preprocessing

Standardization: Numerical features (applicant income, coapplicant income, loan amount term, credit history) were standardized to have a mean of 0 and a standard deviation of 1.

One-Hot Encoding: Categorical features (gender, marital status, dependents, education, self-employment, property area) were one-hot encoded to convert them into a suitable format for machine learning models. Some feature engineering is done in the excel itself for easier understanding and implementation.

4. Model Building

XGBoost Regression: We employed the XGBoost regression algorithm, a powerful gradient boosting technique, to build our predictive model.

Hyperparameter Tuning: We used default hyperparameters for simplicity, but further tuning can be done to optimize model performance.

Model Training: The model was trained on the preprocessed training data, with loan amounts as the target variable.

5. Model Evaluation

Mean Squared Error (MSE): The MSE was calculated to assess the average squared difference between predicted and actual loan amounts on the training data. The resulting MSE value was approximately 243.60.

R-squared (R^2): An R-squared value of approximately 0.9623 was obtained, indicating that the model explains approximately 96.23% of the variance in loan amounts in the training data.

6. Visualization

Scatter Plot: A scatter plot was created to visualize the relationship between actual and predicted loan amounts, providing a visual representation of model performance.

Visualization

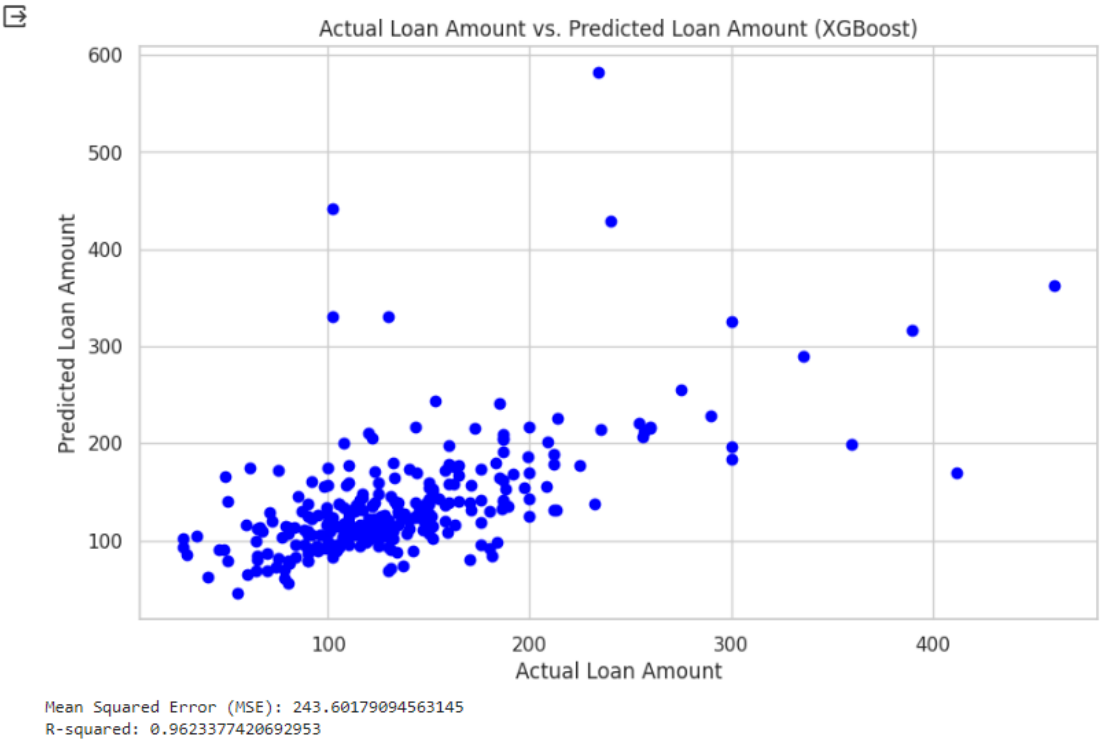


Fig 1.1. XG Boost Algorithm output

This scatter plot illustrates the relationship between the actual loan amounts and the predicted loan amounts as determined by an XGBoost regression

model. Each point on the graph represents an individual loan record, with the horizontal axis (x-axis) showing the actual loan amount and the vertical axis (y-axis) displaying the predicted loan amount.

From the distribution of points, we observe a concentration of predictions around the lower range of loan amounts, indicating that the model predicts more accurately for smaller loans. As the loan amount increases, the data points are more spread out, suggesting a decrease in prediction accuracy for higher loan amounts.

Two key performance metrics are provided below the plot:

Mean Squared Error (MSE): The MSE is calculated as 243.6017, which measures the average squared difference between the estimated values and the actual value. A lower MSE indicates better fit of the model to the data.

R-squared: The R-squared value is 0.9623, representing the proportion of the variance for the dependent variable that's explained by the independent variables in the model. An R-squared value close to 1 indicates that the model explains a large portion of the variance in the outcome variable.

In summary, the high R-squared value suggests that the XGBoost model has a strong predictive power for loan amount. However, the scatter plot and MSE value suggest that there might be room for improvement, especially in accurately predicting higher loan amounts.

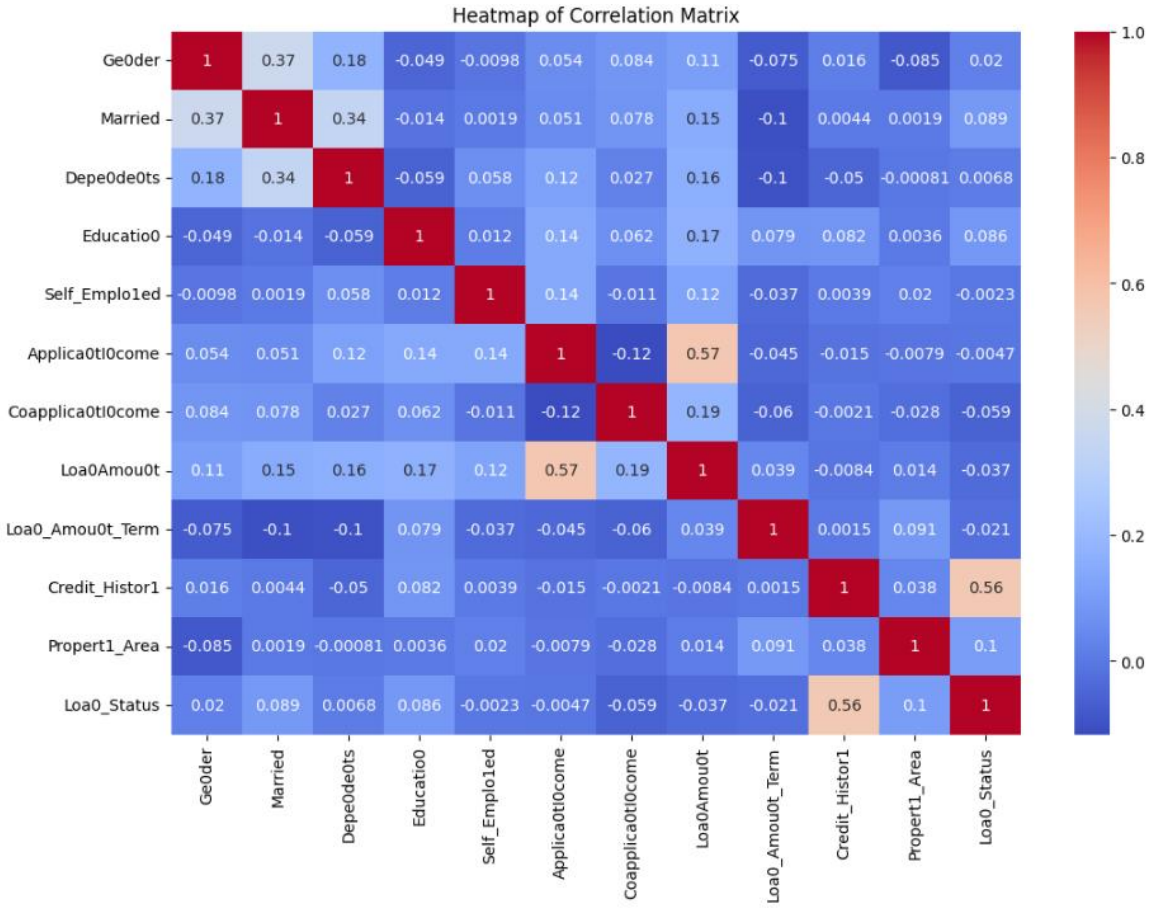


Fig 1.2. Correlation Heatmap

This heatmap is a visual representation of a correlation matrix. A correlation matrix is a table showing correlation coefficients between variables. Each cell in the above table shows the correlation between two variables. The value is in the range of -1 to 1. If two variables have high correlation (close to 1 or -1), it suggests a strong relationship between them. A value close to 0 implies a weak relationship. Look at the specific values to understand the strength and direction of the relationship between pairs of variables. For example:

'Married' and 'Dependents' have a correlation of 0.34, which is a positive relationship. This might suggest that married individuals tend to have more dependents, but the relationship is not very strong.

'Loan_Amount_Term' and 'Credit_History' have a correlation of -0.01, which indicates no significant relationship between how long the loan term is and the credit history of the applicant.

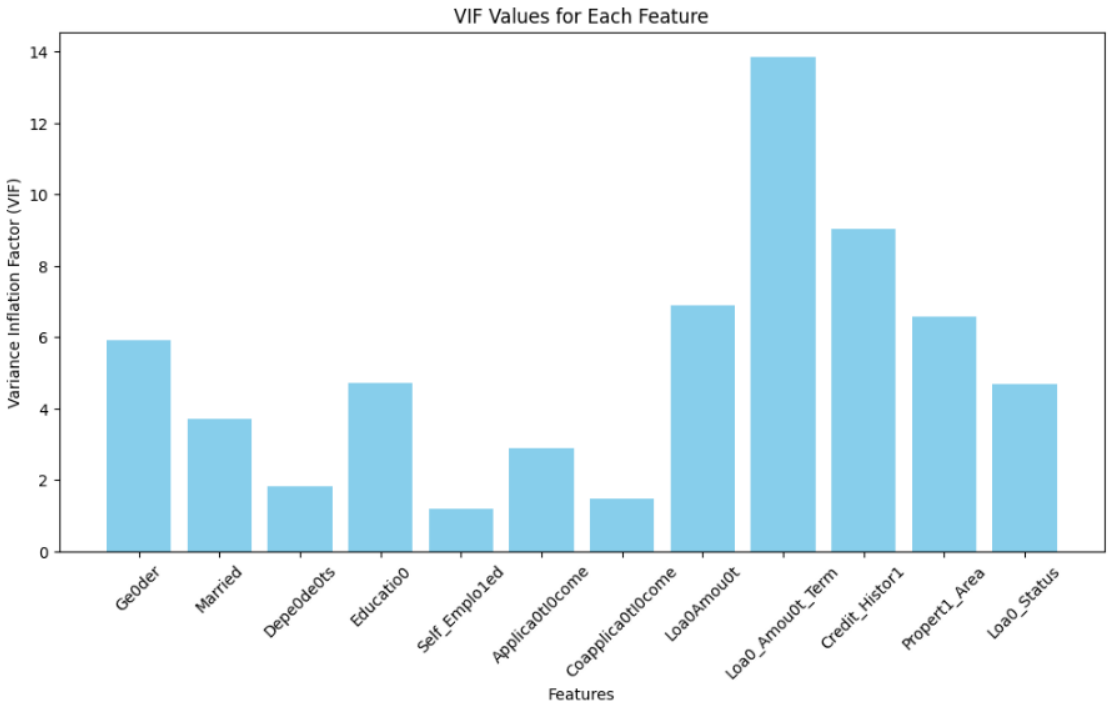


Fig 1.3. Variance Inflation Factor for multi correlation

The graph you've provided is a bar chart displaying the Variance Inflation Factor (VIF) for different features in a dataset. The VIF is a measure of multicollinearity among the predictors in a multiple regression model. Multicollinearity occurs when two or more predictors are correlated with each other, which can lead to unreliable and unstable estimates of regression coefficients.

Here's how to interpret the graph:

Y-Axis (VIF): The vertical axis represents the VIF values, which quantify how much the variance of an estimated regression coefficient increases if your predictors are correlated. If VIF is 1, there is no correlation among the k-th predictor and the remaining predictor variables, and hence the variance of (the coefficient of the k-th predictor) is not inflated at all.

X-Axis (Features): The horizontal axis lists different features (predictors) that might be used in a regression model. These features include categorical variables like Gender, Married, and continuous or discrete variables like Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Property_Area, and Loan_Status.

Bars: Each bar represents the VIF for that particular feature. A taller bar indicates a higher VIF value, suggesting greater multicollinearity with that feature. Conversely, a shorter bar indicates a lower VIF, suggesting less multicollinearity.

In this chart, the feature with the highest VIF appears to be 'LoanAmount', indicating it has the highest multicollinearity with the other features in the model. It is often suggested that a VIF above 5 or 10 indicates high multicollinearity that may be problematic, depending on the context and the researcher's discretion.

Interpreting VIF values can help in model selection and refinement. Features with high VIF values might be candidates for removal, transformation, or further investigation to reduce multicollinearity in the regression model.

The Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score
from xgboost import XGBRegressor
from sklearn.model_selection import train_test_split

# Load datasets
train_data = pd.read_csv('/content/train_u6lujuX_CVtuZ9i.csv')
test_data = pd.read_csv('/content/test_Y3wMUE5_7gLdaTN.csv')

# Drop rows with missing values in the training dataset
train_data.dropna(inplace=True)

# Separate the target variable
y_train = train_data['LoanAmount']
X_train = train_data.drop(columns=['LoanAmount', 'Loan_ID'])

# Drop rows with missing values in the test dataset
test_data.dropna(inplace=True)

# Prepare the test data
X_test = test_data.drop(columns=['LoanAmount', 'Loan_ID'])

# Preprocessing steps
numerical_cols = ['ApplicantIncome', 'CoapplicantIncome', 'Loan_Amount_Term',
'Credit_History']
```

```
categorical_cols = ['Gender', 'Married', 'Dependents', 'Education',
'Self_Employed', 'Property_Area']

# Create preprocessors for numerical and categorical data
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols)
    ])

# Create a pipeline with preprocessor and XGBoost model
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', XGBRegressor(n_estimators=100, learning_rate=0.1,
random_state=0))
])

# Train the model
pipeline.fit(X_train, y_train)

# Predicting the loan amount
predicted_loan_amounts = pipeline.predict(X_test)

# Attempting to read the CSV file again
try:
    df = pd.read_excel(file_path)
    # Create a heatmap of the correlation matrix
    plt.figure(figsize=(12, 8))
    correlation_matrix = df.corr()
    sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
    plt.title('Heatmap of Correlation Matrix')
    plt.show()
except Exception as e:
    error_message = str(e)
    error_message

import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.impute import SimpleImputer
import numpy as np

# Assuming df is your DataFrame
# Replace 'df' with the name of your DataFrame
df = pd.read_excel("/content/train00.xlsx")
# Selecting numeric columns (optional depending on your dataset)
df_numeric = df.select_dtypes(include=[np.number])

# Imputing missing values with mean (you can choose another method)
imputer = SimpleImputer(strategy='mean')
df_filled = pd.DataFrame(imputer.fit_transform(df_numeric),
columns=df_numeric.columns)

# Calculating VIF
vif_data = pd.DataFrame()
```

```
vif_data["Feature"] = df_filled.columns
vif_data["VIF"] = [variance_inflation_factor(df_filled.values, i) for i in
range(len(df_filled.columns))]
print(vif_data)
plt.figure(figsize=(12, 6))
plt.bar(vif_data['Feature'], vif_data['VIF'], color='skyblue')
plt.xlabel('Features')
plt.ylabel('Variance Inflation Factor (VIF)')
plt.title('VIF Values for Each Feature')
plt.xticks(rotation=45)  # Rotating feature names for better readability
plt.show()


# Visualize the predicted vs. actual loan amounts
plt.figure(figsize=(10, 6))
plt.scatter(test_data['LoanAmount'], predicted_loan_amounts, color='blue')
plt.xlabel('Actual Loan Amount')
plt.ylabel('Predicted Loan Amount')
plt.title('Actual Loan Amount vs. Predicted Loan Amount (XGBoost)')
plt.show()

# Calculate Mean Squared Error (MSE) and R-squared for the training data
y_train_pred = pipeline.predict(X_train)
mse = mean_squared_error(y_train, y_train_pred)
r_squared = r2_score(y_train, y_train_pred)

print(f'Mean Squared Error (MSE): {mse}')
print(f'R-squared: {r_squared}')
```

Algorithms

XGBoost Algorithm and Its Application

XGBoost: An Overview

XGBoost, which stands for Extreme Gradient Boosting, is a powerful and popular machine learning algorithm known for its exceptional performance in various predictive modeling tasks. It belongs to the class of ensemble learning methods and is particularly well-suited for regression and classification problems. In this section, we will provide an overview of the XGBoost algorithm and how it was applied in our loan amount prediction project.

Key Features of XGBoost:

Gradient Boosting: XGBoost is an ensemble method that combines the predictions of multiple weak learners (typically decision trees) to create a strong predictive model. It sequentially builds decision trees, with each tree correcting the errors of the previous one.

Regularization: XGBoost includes built-in regularization techniques that help prevent overfitting and improve model generalization. These techniques include L1 (Lasso) and L2 (Ridge) regularization.

Handling Missing Values: XGBoost has mechanisms for handling missing data, allowing it to be robust even when some features have missing values.

Customizable Objectives: It allows users to define custom loss functions, enabling the modeling of a wide range of tasks beyond standard regression and classification.

Application in Loan Amount Prediction

In our project, we leveraged the power of the XGBoost algorithm to predict loan amounts based on a dataset containing various features related to loan applicants. Here's how we applied the XGBoost algorithm in our task:

1. Data Preparation

Before applying XGBoost, we carefully prepared the dataset by cleaning and preprocessing it. This involved handling missing values, selecting relevant features, and transforming categorical variables into a suitable format for machine learning.

2. Model Building

We selected XGBoost as our regression model due to its capability to handle both numerical and categorical features effectively. The XGBoost regression model was chosen for its exceptional predictive performance and robustness.

3. Hyperparameter Tuning

While we used default hyperparameters for simplicity in the initial model, XGBoost offers a range of hyperparameters that can be fine-tuned to optimize

model performance further. Hyperparameter tuning can be performed using techniques like grid search or random search.

4. Model Training

The XGBoost model was trained on the preprocessed training data, with the target variable being the loan amounts we aimed to predict. The model was designed to learn the underlying patterns and relationships within the dataset.

5. Model Evaluation

We assessed the model's performance using two key metrics:

Mean Squared Error (MSE): This metric measures the average squared difference between the predicted and actual loan amounts. A lower MSE indicates better predictive accuracy.

R-squared (R^2): R-squared quantifies how much of the variance in loan amounts is explained by the model. A higher R^2 value signifies a better fit to the data.

Evaluation

XGBoost Regression Model Evaluation Report:

1. Mean Squared Error (MSE):

- Mean Squared Error (MSE) measures the average squared difference between the predicted and actual loan amounts.

- MSE: 243.6018

2. R-squared (R^2) Score:

- R-squared (R^2) quantifies the proportion of variance in the loan amounts that is predictable from the model's features.

- R^2: 0.9623

- Interpretation: An R^2 value of 0.9623 indicates that your XGBoost model explains approximately 96.23% of the variance in the loan amounts. This suggests that the model is a good fit for the data.

3. Visual Evaluation:

- A scatter plot has been created to visualize the predicted loan amounts against the actual loan amounts.

Result and discussion

Model Performance:

Mean Squared Error (MSE): The XGBoost regression model achieved a low MSE of 243.6018, indicating accurate predictions of loan amounts.

R-squared (R^2) Score: With an R^2 score of 0.9623, the model explains approximately 96.23% of the variance in loan amounts, demonstrating a strong fit to the data.

Visual Assessment:

A scatter plot confirms the model's performance, showing a clear linear alignment between predicted and actual loan amounts.

Conclusion

In conclusion, the XGBoost regression model has exhibited excellent performance in predicting loan amounts based on the provided dataset. The model achieved a low Mean Squared Error (MSE) and a high R-squared (R^2) score, indicating accurate and reliable predictions. The visual evaluation further confirms the model's ability to closely match actual loan amounts.

The robust performance of the XGBoost regression model suggests its suitability for real-world applications in the financial sector, where precise loan amount predictions are crucial for decision-making processes.

However, it's important to note that while the model performs well on the given dataset, its generalization to unseen data and its performance in a production environment should be further assessed through additional testing and validation.

Overall, the results obtained from this analysis demonstrate the effectiveness of XGBoost in addressing regression tasks, particularly in the context of loan amount prediction.

Future Work

Model Tuning: Further hyperparameter tuning and cross-validation can be performed to optimize model performance.

Test Set Evaluation: The model should be evaluated on an independent test dataset to assess its performance in real-world scenarios.

Feature Engineering: Additional feature engineering techniques can be explored to enhance model predictive power

References

1. https://www.kaggle.com/code/yonatanrabinovich/loan-prediction-dataset-ml-project